

Ilm-d on Intel XPU/GPU: Kubernetes Deployment Whitepaper

Contents

Intel XPU Deployment Whitepaper: Ilm-d	4
Table of Contents	4
Scope note: which guides actually ship an Intel XPU configuration	5
Chapter 0. Common Installation	5
0.1 Cluster and Hardware Requirements	5
0.1.1 Install the Intel GPU DRA Driver	5
0.2 Clone the Repository	6
0.3 Install the Gateway API Inference Extension CRDs	6
0.4 Namespace and HuggingFace Token	6
0.5 Proxy Configuration (if required)	7
1.1 Optimized Baseline	7
Overview	7
Prerequisites	7
Deployment Steps	7
Verification	8
Inference Test	8
Troubleshooting	8
Cleanup	8
Configuration Reference	8
1.2 P/D Disaggregation	9
Overview	9
Prerequisites	9
Deployment Steps	9
Verification	10
Inference Test	10
Troubleshooting	10
Cleanup	10
Configuration Reference	11
1.3 Precise Prefix Cache Routing	11
Overview	11
Prerequisites	11
Deployment Steps	12
Verification	12
Inference Test	12
Troubleshooting	13
Cleanup	13

Configuration Reference	13
1.4 Tiered Prefix Cache	13
Overview	13
Prerequisites	14
Deployment Steps	14
Verification	14
Inference Test	14
Troubleshooting	15
Cleanup	15
Configuration Reference	15
1.5 Wide Expert Parallelism	15
Overview	15
Prerequisites	16
Deployment Steps	16
Verification	16
Inference Test	16
Troubleshooting	17
Cleanup	17
Configuration Reference	17
1.6 Multimodal Serving (Aggregated)	17
Overview	17
Prerequisites	17
Deployment Steps	18
Verification	18
Inference Test	18
Troubleshooting	18
Cleanup	19
Configuration Reference	19
2.1 Flow Control	19
Overview	19
Prerequisites	19
Deployment Steps	20
Verification	20
Inference Test	20
Troubleshooting	21
Cleanup	21
Config Reference	21
2.2 Predicted Latency-Based Routing	22
Overview	22
Prerequisites	22
Deployment Steps	22
Verification	23
Inference Test	23
Troubleshooting	24
Cleanup	24
Config Reference	24
2.3 Workload Autoscaling	25
Overview	25
Prerequisites	25

Deployment Steps (KEDA + EPP Metrics path)	26
Verification	26
Inference Test	26
Troubleshooting	27
Cleanup	27
Config Reference	27
2.4 Rollouts	27
Overview	28
Prerequisites	28
Deployment Steps	28
Rolling Update	28
Blue-Green Update — example: Intel XPU node/accelerator migration	28
LoRA Adapter Rollout	29
Verification	29
Inference Test	30
Troubleshooting	30
Cleanup	30
Config Reference	30
3.1 Agentic Serving	31
Overview	31
Prerequisites	31
Deployment Steps (Intel XPU — compose the building blocks manually)	31
Verification	32
Inference Test	32
Troubleshooting	32
Cleanup	33
Config Reference	33
3.2 Multi-Model Routing	33
Overview	33
Prerequisites	33
Deployment Steps	34
Verification	34
Inference Test	34
Advanced: LoRA Adapter Routing	35
Troubleshooting	35
Cleanup	35
Config Reference	36
4.1 Asynchronous Processing	36
Overview	36
Prerequisites	36
Deployment Steps	36
Verification	37
Inference Test	37
Troubleshooting	37
Cleanup	37
Config Reference	38
4.2 Batch Gateway	38
Overview	38
Prerequisites	38
Deployment Steps	38

Verification	39
Inference Test	39
Troubleshooting	40
Cleanup	40
Config Reference	40
5.1 No-Kubernetes Deployment	40
Overview	40
What Needs to Change for Intel XPU	41
Prerequisites	41
Deployment Steps	42
Verification	43
Inference Test	43
Troubleshooting	43
Cleanup	43
Config Reference	43
5.2 Encode Disaggregation (Multimodal)	44
Overview	44
Why this guide has no Intel XPU deployment steps	44
Appendix A. Environment Variable Reference	44
Appendix B. Known Issues	45

Intel XPU Deployment Whitepaper: Ilm-d

Target hardware: Intel Data Center GPU (Max series, Arc Pro B60) exposed to Kubernetes via the `gpu.intel.com` Dynamic Resource Allocation (DRA) device class

Table of Contents

- 00-common-installation.md — Chapter 0. Common Installation (applies to every case below)
- **Chapter 1. Intel XPU Well-Lit Paths** (real Intel XPU overlay confirmed in-repo)
 - 01-optimized-baseline.md — 1.1 Optimized Baseline
 - 02-pd-disaggregation.md — 1.2 P/D Disaggregation
 - 03-precise-prefix-cache-routing.md — 1.3 Precise Prefix Cache Routing
 - 04-tiered-prefix-cache.md — 1.4 Tiered Prefix Cache
 - 05-wide-ep.md — 1.5 Wide Expert Parallelism
 - 06-multimodal-serving.md — 1.6 Multimodal Serving (Aggregated)
- **Chapter 2. Router-Layer Features** (hardware-agnostic, compose on top of Chapter 1)
 - 07-flow-control.md — 2.1 Flow Control
 - 08-predicted-latency-routing.md — 2.2 Predicted Latency-Based Routing
 - 09-workload-autoscaling.md — 2.3 Workload Autoscaling
 - 10-rollouts.md — 2.4 Rollouts (note: repo path moved from `guides/rollouts/` to `docs/operations/rollouts/`)
- **Chapter 3. Composite Workloads**
 - 11-agentic-serving.md — 3.1 Agentic Serving (Intel XPU: manual composition of Chapter 1 guides, no ready-made upstream deployment)
 - 12-multi-model-routing.md — 3.2 Multi-Model Routing
- **Chapter 4. Experimental Guides**
 - 13-asynchronous-processing.md — 4.1 Asynchronous Processing

- 14-batch-gateway.md — 4.2 Batch Gateway
- **Chapter 5. Other Deployment Forms**
 - 15-no-kubernetes-deployment.md — 5.1 No-Kubernetes Deployment (NVIDIA-only upstream — this chapter documents the Intel XPU substitution)
 - 16-encode-disaggregation.md — 5.2 Encode Disaggregation (Multimodal) (no Intel XPU backend exists upstream)
- appendix-a-env-vars.md — Appendix A. Environment Variable Reference
- appendix-b-known-issues.md — Appendix B. Known Issues

tracking.md in this directory is a separate, continuously-updated per-case validation-status tracker (hands-on-verified vs. documented-but-untested) — useful when browsing the repository directly, but it is not one of the chapters above and is not included in the compiled PDF/Word export of this whitepaper.

Scope note: which guides actually ship an Intel XPU configuration

A repo-wide search of guides/**/*.*md and guides/**/*.*yaml for xpu/intel found **dedicated Kustomize overlays** (modelserver/xpu/...) in exactly six guides: optimized-baseline, pd-disaggregation, precise-prefix-cache-routing, tiered-prefix-cache, wide-ep-lws, and multimodal-serving/aggregation. These are Chapter 1.

The remaining guides (flow-control, predicted-latency-routing, workload-autoscaling, rollouts, agentic-serving, multi-model-routing, asynchronous-processing, batch-gateway, no-kubernetes-deployment) are **router-layer or composite features that are accelerator-agnostic by design** — they sit on top of whatever model-server overlay you deploy (including the Intel XPU overlay from optimized-baseline), and reference it rather than shipping a separate XPU overlay of their own (Chapters 2–5). no-kubernetes-deployment is explicitly documented as NVIDIA + vLLM specific, with the Intel XPU substitution spelled out in full in its chapter — it is not a maintained, first-class Intel XPU path upstream. encode-disaggregation (under multimodal-serving/e-disaggregation/) has **no** Intel XPU backend at all — its “Supported Hardware Backends” table lists NVIDIA GPU only. All case-by-case verification status is tracked in tracking.md.

Chapter 0. Common Installation

Applies to every case in this whitepaper. Later chapters link back here instead of repeating these steps.

0.1 Cluster and Hardware Requirements

- A Kubernetes cluster (1.29+ recommended; 1.34+ if you plan to use the DRA-based deployment templates, since DRA’s structured-parameters API only reached GA in that release) with nodes exposing Intel Data Center GPU Max series or Intel Arc Pro GPUs, and the host Linux Intel GPU kernel driver already installed on each node
- The **Intel GPU DRA driver** installed (see 0.1.1 below), so that `kubectl get deviceclass` lists `gpu.intel.com`
- Local client tools: `kubectl`, `helm` (v3); see `helpers/client-setup/README.md`

0.1.1 Install the Intel GPU DRA Driver

Not part of the llm-d repo itself — this is a cluster-level, vendor-provided component from `intel/intel-resource-drivers-for-kubernetes` that must be installed once per cluster before any Intel XPU case in this whitepaper. Install via its published Helm chart:

```
helm install \
  --namespace intel-gpu-resource-driver \
  --create-namespace \
  intel-gpu-resource-driver oci://ghcr.io/intel/intel-resource-drivers-for-
  ↪ kubernetes/intel-gpu-resource-driver-chart
```

Verify the driver is up and the `gpu.intel.com DeviceClass` and per-node `ResourceSlice` objects were published:

```
kubectl get pods -n intel-gpu-resource-driver
kubectl get deviceclass gpu.intel.com
kubectl get resourceslices
```

See the driver's own GPU documentation for container-runtime CDI requirements (CRI-O 1.23+ or Containerd 1.7+ with CDI enabled) and alternative install methods (kustomize, Node Feature Discovery-scoped rollout).

0.2 Clone the Repository

```
export branch="main"
git clone https://github.com/llm-d/llm-d.git && cd llm-d && git checkout
  ↪ ${branch}
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
```

`guides/env.sh` centralizes the chart versions and OCI addresses shared by every guide (`GAIE_VERSION`, `ROUTER_CHART_VERSION`, `ROUTER_STANDALONE_CHART`, etc.), so commands never hardcode a version.

0.3 Install the Gateway API Inference Extension CRDs

[!IMPORTANT] CRD installation is a single command below. Gateway Provider installation (Istio / GKE / Agentgateway) lives under `docs/infrastructure/gateway/`.

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api-inference-
  ↪ extension/releases/download/${GAIE_VERSION}/v1-manifests.yaml
```

If you plan to use a Kubernetes Gateway (instead of the Router's standalone mode), follow the provider-specific guide under `${REPO_ROOT}/docs/infrastructure/gateway/` once. A single Gateway can be shared across every case in this whitepaper via a dedicated `HTTPRoute` per case.

0.4 Namespace and HuggingFace Token

```
export NAMESPACE="llm-d-<case-name>"
kubectl create namespace ${NAMESPACE} --dry-run=client -o yaml | kubectl apply
  ↪ -f -
```

```
export HF_TOKEN=<your HuggingFace token>
kubectl create secret generic llm-d-hf-token \
  --from-literal="HF_TOKEN=${HF_TOKEN}" \
  --namespace "${NAMESPACE}" \
  --dry-run=client -o yaml | kubectl apply -f -
```

0.5 Proxy Configuration (if required)

If the cluster needs a corporate proxy to reach the HuggingFace Hub, add HTTP_PROXY / HTTPS_PROXY / NO_PROXY (and lowercase equivalents) to the relevant container env. This is applied via a Kustomize patch in the overlay-based cases in this whitepaper, or via --set / a values file in Helm-based steps.

1.1 Optimized Baseline

Overview

Deploys the recommended out-of-the-box routing configuration for vLLM/SGLang/TensorRT-LLM: prefix-cache-aware scoring (via the prefix-cache-scorer and no-hit-lru-scorer) combined with load-aware scoring (kv-cache-utilization and queue-size scorers). This is the baseline that several other guides (Flow Control, Predicted Latency Routing, Agentic Serving) build on top of. This guide has a dedicated **Intel XPU E2E CI workflow** (consolidate-status-optimized-baseline- intel-acc-xpu-vllm-x.yaml), making it the most CI-validated Intel XPU path in the repo today.

Default model: Qwen/Qwen3-32B (GPU); the Intel XPU overlay uses a CI-sized Qwen/Qwen3-0.6B for fast validation — swap in a production model before real deployment.

Prerequisites

Chapter 0 only. No case-specific prerequisites.

Deployment Steps

Step 1 — Deploy the llm-d Router (Standalone mode)

```
export GUIDE_NAME="optimized-baseline"
export NAMESPACE="llm-d-optimized-baseline"

helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/${GUIDE_NAME}.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

Step 2 — Deploy the Model Server (Intel XPU overlay)

[!IMPORTANT] The Intel XPU path uses Kubernetes **DRA**: GPUs are declared via a ResourceClaimTemplate (deviceClassName: gpu.intel.com), not resources.limits. Confirm the cluster has DRA enabled and the gpu.intel.com device class available before deploying.

```
export ACCELERATOR_TYPE=xpu
export MODEL_SERVER=vllm
kubectl apply -n ${NAMESPACE} -k ${REPO_ROOT}/guides/${GUIDE_NAME} \
  ↪ /modelserver/${ACCELERATOR_TYPE}/${MODEL_SERVER}/
```

[!NOTE] Unlike the NVIDIA GPU path, the Intel XPU overlay path has **no** \${INFRA_PROVIDER} suffix — apply the directory directly.

This overlay (modelserver/xpu/vllm/): - uses recipes/modelserver/base/single-host/default as its base and the recipes/modelserver/components/images/xpu-vllm image component - patches the decode Deployment (patch-vllm.yaml) to run vllm serve Qwen/Qwen3-0.6B --dtype=float16 --disable-sliding-window ... - adds a ResourceClaimTemplate (resource-claim-template.yaml) requesting 1 gpu.intel.com device per replica, with fsGroup: 107 / supplementalGroups: [107] for Intel GPU device node access

Step 3 — (Optional) Enable Monitoring

```
kubectl apply -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/recipes/modelserver/components/monitoring
```

Verification

```
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
↳ jsonpath='{.spec.clusterIP}')
kubectl get pods -n ${NAMESPACE}
```

Inference Test

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace="${NAMESPACE}" \
  --env="IP=${IP}" \
  -- /bin/bash

curl -X POST http://${IP}/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "Qwen/Qwen3-0.6B", "prompt": "How are you today?"}' | jq
```

Troubleshooting

- **ResourceClaim stuck Pending:** confirm the node has registered the gpu.intel.com device class; `kubectl describe resourceclaim <name> -n ${NAMESPACE}`.
- **vLLM usage-telemetry write failures under restricted SecurityContext** (e.g. OpenShift): set `DO_NOT_TRACK=1` in the container env (already applied in the precise-prefix-cache-routing XPU overlay; port the same fix here if you hit it).

Cleanup

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
kubectl delete -n ${NAMESPACE} -k ${REPO_ROOT}/guides/${GUIDE_NAME}
↳ /modelserver/${ACCELERATOR_TYPE}/${MODEL_SERVER}/
kubectl delete namespace ${NAMESPACE}
```

Configuration Reference

Parameter	Value	Purpose
--dtype	float16	Intel XPU overlay dtype
--disable-sliding-window	set	Required by the CI-sized model
deviceClassName	gpu.intel.com	DRA device class

Parameter	Value	Purpose
fsGroup / supplementalGroups	107	Intel GPU render-node group access

Resource requirements (Intel XPU overlay, Qwen/Qwen3-0.6B, 2 replicas):

Role	Replicas	CPU (req/limit)	Memory (req/limit)	GPU
Decode	2	4 / 8 cores	12Gi / 24Gi	1× gpu.intel.com per replica (DRA claim)

1.2 P/D Disaggregation

Overview

Splits inference into independently-scaled Prefill and Decode deployments, connected via NIXL KV transfer. Native to the llm-d Router, so it composes with prefix-cache-aware and load-aware routing from Optimized Baseline. Default GPU example uses openai/gpt-oss-120b (8× TP=1 Prefill + 2× TP=4 Decode); the Intel XPU overlay uses the smaller Qwen/Qwen3-0.6B for fast validation.

Best suited for: medium-large models, long input sequences (e.g. 10k ISL / 1k OSL), sparse MoE architectures.

Prerequisites

Chapter 0 only.

Deployment Steps

Step 1 — Deploy the llm-d Router (Standalone mode)

```
export GUIDE_NAME="pd-disaggregation"
export NAMESPACE="llm-d-pd-disaggregation"
export MODEL_NAME="Qwen/Qwen3-0.6B" # Intel XPU overlay default; swap for
↪ production models
```

```
helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/${GUIDE_NAME}.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

Step 2 — Deploy the Model Server (Intel XPU overlay)

[!IMPORTANT] GPUs are declared via DRA ResourceClaimTemplate (deviceClassName: gpu.intel.com), not resources.limits."gpu.intel.com/xen".

```
kubectl apply -n ${NAMESPACE} -k
↪ ${REPO_ROOT}/guides/pd-disaggregation/modelserver/xpu/vllm
```

This overlay includes:

- patch-prefill.yaml / patch-decode.yaml: vLLM launch args, including --kv-transfer-config with kv_connector: NixlConnector, kv_role: kv_both, kv_buffer_device: cpu; and env vars VLLM_USE_V1=1, VLLM_NIXL_SIDE_CHANNEL_HOST (from status.podIP), VLLM_NIXL_SIDE_CHANNEL_PORT=5600, UCX_TLS=tcp, VLLM_WORKER_MULTIPROC_METHOD=spawn
- resource-claim-templates.yaml: two ResourceClaimTemplates, xpu-prefill-claim and xpu-decode-claim, each requesting 1 gpu.intel.com device

For clusters with RDMA connectivity, use modelserver/xpu/vllm-rdma instead (UCX transport ib,rc,ze_copy).

Step 3 — (Optional) Enable Monitoring

```
kubectl apply -n ${NAMESPACE} -k
↪ ${REPO_ROOT}/guides/recipes/modelserver/components/monitoring-pd
```

Verification

```
kubectl get pods -n ${NAMESPACE}
kubectl get pods -n ${NAMESPACE} -l llm-d.ai/role=decode -w
kubectl get pods -n ${NAMESPACE} -l llm-d.ai/role=prefill -w

export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
↪ jsonpath='{.spec.clusterIP}')
```

Inference Test

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace=${NAMESPACE} \
  --env="IP=${IP}" \
  -- /bin/bash

curl -X POST http://${IP}/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "Qwen/Qwen3-0.6B", "prompt": "How are you today?"}' | jq
```

Troubleshooting

- **Pod stuck at Init:0/1:** known routing-proxy sidecar issue (llm-d/llm-d#241); workaround: set proxy.enabled: false — the Gateway API + InferencePool routing already covers what the sidecar provided.
- **KV transfer failures:** check decode/prefill container logs for nixl: `kubectl logs -n ${NAMESPACE} <pod> -c modelserver | grep -i nixl`.
- **ResourceClaim stuck Pending:** confirm the gpu.intel.com device class is registered; `kubectl get resourceclaims -n ${NAMESPACE}` for details.
- **HuggingFace download failures:** check proxy env vars and DNS (nslookup huggingface.co).

Cleanup

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
```

```
kubectl delete -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/pd-disaggregation/modelserver/xpu/vllm
```

Configuration Reference

Parameter	Value	Purpose
kv_connector	NixlConnector	KV transfer connector
kv_role	kv_both	Both send and receive
kv_buffer_device	cpu	Reduces GPU memory pressure
VLLM_USE_V1	1	Required for P/D
VLLM_WORKER_MULTIPROC_METHOD	spawn	Multiprocess start method
UCX_TLS	tcp (or ib, rc, ze_copy for the RDMA overlay)	UCX transport

Resource requirements (Intel XPU overlay, Qwen/Qwen3-0.6B):

Role	Replicas	CPU	Memory	GPU
Decode	1	16 cores	64Gi	1× gpu.intel.com (DRA claim)
Prefill	3	16 cores/replica	64Gi/replica	1× gpu.intel.com/replica (DRA claim)

[!WARNING] Production models (e.g. openai/gpt-oss-120b) require substantially more CPU/memory/GPU than the table above — re-plan capacity for your target model.

1.3 Precise Prefix Cache Routing

Overview

Routes on precise, per-pod KV-cache state instead of traffic heuristics. Each vLLM pod publishes KV-cache events over ZMQ; the router indexes them by block hash and scores candidates by resident prefix fraction, combined with load-aware scoring. Intel XPU overlay uses the CI-sized Qwen/Qwen3-0.6B model; update the router's token-producer modelName in router/precise-prefix-cache-routing.values.yaml to match whatever model you deploy — precise scoring only works if the two match.

Prerequisites

Chapter 0 only. Note --block-size in the vLLM args must match the scorer's tokenProcessorConfig.blockSize (default 64).

Deployment Steps

Step 1 — Deploy the llm-d Router

```
export GUIDE_NAME="precise-prefix-cache-routing"
export NAMESPACE="llm-d-precise-prefix-cache-routing"
```

```
helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/${GUIDE_NAME}.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

[!NOTE] The router defaults to **active-active HA** (2 EPP replicas), each subscribing to every vLLM pod so both indexes converge independently. Set `--set router.epp.replicas=1` for small fleets or smoke tests.

Step 2 — Deploy the Model Server (Intel XPU overlay)

```
kubectl apply -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/precise-prefix-cache-routing/modelserver/xpu/vllm
```

Key args baked into the overlay's `patch-vllm.yaml`:

```
args:
- "Qwen/Qwen3-0.6B"
- "--dtype=float16"
- "--disable-sliding-window"
- "--block-size=64"
- "--kv-events-config"
- '
↳ '{"enable_kv_cache_events":true,"publisher":"zmq","endpoint":"$(KV_EVENTS_ENDPOINT)","0.6B"}'
env:
- name: KV_EVENTS_ENDPOINT
  value: "tcp://*:5556"           # pod-discovery mode: every router replica
↳ dials each pod directly
- name: DO_NOT_TRACK
  value: "1"                     # disables vLLM telemetry writes to /.config,
↳ which fail under              # restricted SecurityContexts (e.g. OpenShift)
```

Port 5556 (kv-events) is exposed on the pod so every router replica can reach the per-pod ZMQ socket.

Verification

```
kubectl get pods -n ${NAMESPACE}
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
↳ jsonpath='{.spec.clusterIP}')
```

Inference Test

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace=${NAMESPACE} \
  --env="IP=${IP}" \
  -- /bin/bash
```

```
curl -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-d '{"model": "Qwen/Qwen3-0.6B", "prompt": "How are you today?"}' | jq
```

Troubleshooting

- **Scores look wrong / no cache hits registered:** confirm router/precise-prefix-cache- routing.values.yaml's modelName matches the model actually deployed by the overlay.
- **vLLM telemetry write errors under OpenShift-style restricted SecurityContext:** confirm DO_NOT_TRACK=1 is set.
- **Router not receiving KV events:** verify port 5556 is reachable from the router pod to each vLLM pod (kubectl exec into the router pod and test connectivity).

Cleanup

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
kubectl delete -n ${NAMESPACE} -k
↪ ${REPO_ROOT}/guides/${GUIDE_NAME}/modelserver/xpu/vllm
```

Configuration Reference

Parameter	Value	Purpose
--block-size	64	Must match scorer tokenProcessorConfig.blockSize
enable_kv_cache_events	true	Turns on KV-cache event publishing
publisher	zmq	Event transport
KV_EVENTS_ENDPOINT	tcp://*:5556	Pod-discovery mode ZMQ bind address
DO_NOT_TRACK	1	Disables vLLM telemetry (avoids write failures under restricted SCCs)

1.4 Tiered Prefix Cache

Overview

Offloads evicted KV-cache blocks from accelerator memory to larger, cheaper tiers (CPU RAM, and optionally shared filesystem), increasing effective cache size for multi-turn/long-context workloads. Multiple offloading implementations exist (vLLM native OffloadingConnector, LMCache, MooncakeStore, SGLang HiCache); the Intel XPU overlay ships the **vLLM native** and **LMCache connector (CPU tier)** paths.

Prerequisites

Chapter 0, plus the prefix-aware routing from Optimized Baseline (already the default scorer stack used here).

Deployment Steps

Step 1 — Deploy the llm-d Router

```
export GUIDE_NAME="tiered-prefix-cache"
export NAMESPACE="llm-d-tiered-prefix-cache"

helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/${GUIDE_NAME}.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

Step 2 — Deploy the Model Server (Intel XPU overlay, choose a path)

Native (vLLM OffloadingConnector, CPU RAM tier):

```
kubectl apply -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/tiered-prefix-cache/modelserver/xpu/vllm/base
```

LMCache connector (CPU RAM tier):

```
kubectl apply -n ${NAMESPACE} -k ${REPO_ROOT}/guides/tiered-prefix-
↳ cache/modelserver/xpu/vllm/lmcache-connector/cpu/base
```

[!IMPORTANT] The Intel XPU base overlay sets `securityContext.privileged: true` on the model-server container in addition to the DRA ResourceClaimTemplate (`xpu-vllm-gpu`) — this is required for the offloading connector to manage host-pinned memory. Review your cluster's Pod Security Admission policy before applying.

Verification

```
kubectl get pods -n ${NAMESPACE}
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
↳ jsonpath='{.spec.clusterIP}')
```

Inference Test

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace="${NAMESPACE}" \
  --env="IP=${IP}" \
  -- /bin/bash

curl -X POST http://${IP}/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "Qwen/Qwen3-0.6B", "prompt": "Summarize the benefits of
↳ KV-cache offloading."}' | jq
```

Send a long, repeated-prefix prompt twice to observe a cache hit on the second call (check router logs / metrics for prefix-cache reuse).

Troubleshooting

- **Pod fails to start under Pod Security Admission “restricted” policy:** the overlay requires `privileged: true`; either relax the namespace’s PSA label or use a different offloading path that doesn’t need privileged access (verify per-connector requirements before assuming).
- **No observable cache reuse:** confirm the CPU-tier offload size is large enough for your workload and that requests are actually re-sending the same prefix.

Cleanup

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
kubectl delete -n ${NAMESPACE} -k
  ↪ ${REPO_ROOT}/guides/tiered-prefix-cache/modelserver/xpu/vllm/base
# or, if you deployed LMCache:
kubectl delete -n ${NAMESPACE} -k ${REPO_ROOT}/guides/tiered-prefix-
  ↪ cache/modelserver/xpu/vllm/lmcache-connector/cpu/base
```

Configuration Reference

Parameter	Value	Purpose
deviceClassName	gpu.intel.com	DRA device class (claim name xpu-vllm-gpu)
securityContext.privileged	true	Required for host-pinned memory management
CPU cache offload tier	CPU RAM (native) or CPU RAM (LMCache)	Effective KV-cache size beyond accelerator memory

1.5 Wide Expert Parallelism

Overview

Deploys a wide expert-parallel MoE model with P/D disaggregation using `LeaderWorkerSets` and DP-aware scheduling. The reference GPU configuration targets DeepSeek-R1-0528 across 32 GPUs; the **validated Intel XPU configuration** uses the smaller DeepSeek-V2-Lite-Chat shape:

Parameter	Value
Model	deepseek-ai/DeepSeek-V2-Lite-Chat
Prefill Tensor Parallelism	2
Decode Tensor Parallelism	2
Total XPUs	4
Expert Parallelism	enabled
All2All backend	allgather_reducescatter
KV transfer	NIXL, kv_buffer_device=xpu
UCX transport	tcp, ze_copy (validated non-RDMA configuration)

[!NOTE] The Intel XPU backend uses **XCCL** for collective communication; NIC-ID-based rail-only connectivity configurations (used on some NVIDIA RoCE setups) are not compatible and will fail.

Prerequisites

Chapter 0, plus the Intel GPU DRA driver already covers this case's device requirement — no additional driver install beyond Chapter 0.

Deployment Steps

Step 1 — Deploy the llm-d Router (Standalone mode, with the XPU override)

```
export GUIDE_NAME="wide-ep-lws"
export NAMESPACE="llm-d-wide-ep-lws"

helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/${GUIDE_NAME}.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/xpu.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

[!IMPORTANT] The xpu.values.yaml override is required — without it, EPP does not target the single decode sidecar port exposed by the Intel XPU manifests.

Step 2 — Deploy the Model Server

```
export MODEL=deepseek-ai/DeepSeek-V2-Lite-Chat
kubectl apply -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/wide-ep-lws/modelserver/xpu/vllm
```

Step 3 — (Optional) Enable Monitoring / Topology-Aware Scheduling

Monitoring follows the same pattern as other guides (kubectl apply -k ../guides/recipes/modelserver/pd, if applicable). TAS overlays in this guide are GKE H200/B200-specific and do not apply to the Intel XPU path.

Verification

```
kubectl get pods -n ${NAMESPACE}
kubectl get pods -n ${NAMESPACE} -l llm-d.ai/role=decode -w
kubectl get pods -n ${NAMESPACE} -l llm-d.ai/role=prefill -w
```

Inference Test

```
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
↳ jsonpath='{.spec.clusterIP}')
```

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace=${NAMESPACE} \
  --env="IP=${IP}" \
  -- /bin/bash
```

```
curl -X POST http://${IP}/v1/completions \
```



```
-H 'Content-Type: application/json' \
-d '{"model": "deepseek-ai/DeepSeek-V2-Lite-Chat", "prompt": "Explain
  ↪ expert parallelism."}' | jq
```

Troubleshooting

- **Collective communication failures / hangs:** confirm the cluster is using XCCL, not a NIC-ID/rail-only RoCE configuration — the latter is documented as incompatible with the Intel XPU backend.
- **EPP routing to the wrong port:** confirm `router/xpu.values.yaml` was included in the `helm install` — without it EPP targets the wrong (non-XPU) sidecar port.

Cleanup

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
kubectl delete -n ${NAMESPACE} -k
  ↪ ${REPO_ROOT}/guides/wide-ep-lws/modelserver/xpu/vllm
```

Configuration Reference

Parameter	Value	Purpose
<code>kv_buffer_device</code>	<code>xpu</code>	KV buffer resident on the accelerator (differs from the cpu default used by the PD-disaggregation guide)
All2All backend	<code>allgather_reducescatter</code>	MoE expert-parallel communication pattern
UCX transport	<code>tcp,ze_copy</code>	Validated non-RDMA transport for Intel XPU
Total XPUs	<code>4</code>	2 (prefill TP) + 2 (decode TP), validated shape

1.6 Multimodal Serving (Aggregated)

Overview

Deploys the recommended configuration for multimodal (image + text) vLLM serving with prefix-cache aware routing that matches combined text + image hashes, plus load-aware scoring. Default GPU model is Qwen/Qwen3-VL-32B-Instruct; the Intel XPU overlay targets **Intel Arc Pro B60**.

Prerequisites

Chapter 0 only.

Deployment Steps

Step 1 — Deploy the Llm-d Router

```
export GUIDE_NAME="aggregation"
export NAMESPACE="llm-d-multimodal-aggregation"

helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/multimodal-serving/${GUIDE_NAME}/router/ \
    ↳ ${GUIDE_NAME}.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

Step 2 — Deploy the Model Server (Intel XPU overlay)

```
kubectl apply -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/multimodal-serving/${GUIDE_NAME}/modelserver/xpu/vllm/
```

The overlay patches the decode Deployment's resource requests/limits and DRA claim (intel-claim-template-decode, requesting 1 gpu.intel.com device); the base image and multimodal serving args come from the shared base manifest.

Step 3 — (Optional) Enable Monitoring

```
kubectl apply -n ${NAMESPACE} -k
↳ ${REPO_ROOT}/guides/recipes/modelserver/components/monitoring
```

Verification

```
kubectl get pods -n ${NAMESPACE}
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
↳ jsonpath='{.spec.clusterIP}')
```

Inference Test

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace="${NAMESPACE}" \
  --env="IP=${IP}" \
  -- /bin/bash

curl -X POST http://${IP}/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "Qwen/Qwen3-VL-32B-Instruct",
    "messages": [{"role": "user", "content": [
      {"type": "text", "text": "What is in this image?"},
      {"type": "image_url", "image_url": {"url":
↳ "https://example.com/sample.jpg"}}
    ]}]
  }' | jq
```

Troubleshooting

- **GPU allocation fails:** confirm the node advertises gpu.intel.com resources compatible with Intel Arc Pro B60 and that the DRA driver version supports it.

- **Image decoding errors:** check vLLM logs for multimodal preprocessing errors; verify the image URL is reachable from within the cluster.

Cleanup

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
kubectl delete -n ${NAMESPACE} -k
↪ ${REPO_ROOT}/guides/multimodal-serving/${GUIDE_NAME}/modelserver/xpu/vllm/
```

Configuration Reference

Parameter	Value	Purpose
deviceClassName	gpu.intel.com	DRA device class (claim intel-claim-template-decode)
Target hardware	Intel Arc Pro B60	As documented in the guide's hardware table

2.1 Flow Control

Overview

Hardware-agnostic — same steps apply on Intel XPU as any other accelerator.

Flow Control adds intelligent request queuing at the llm-d Router (EPP) level. Traditional load balancing falls short for LLM inference because resource consumption varies wildly per request; shifting queuing into the Router enables **multi-tenant fairness** (prevent noisy neighbors from starving other tenants) and **no-regret scheduling** (hold requests during saturation instead of committing them to a server's local queue where they get stuck). Source: guides/flow-control/README.md.

Requests are classified by a FlowKey (Fairness ID + Priority). The EPP maintains separate in-memory queues per flow and dispatches by priority band, then fairness across tenants within a band, then arrival order within a flow. Default policies (global-strict-fairness-policy + fcfs-ordering-policy + utilization-detector) intentionally mimic legacy FCFS behavior for a seamless transition — production deployments should switch the saturation detector to concurrency-detector per the guide's tuning.md to avoid telemetry lag risk.

Supported Hardware Backends: Flow Control is a software-level EPP feature and is entirely hardware-agnostic — it supports every accelerator the Optimized Baseline guide supports, including Intel XPU. This guide dynamically reuses whichever model server overlay you deploy from Optimized Baseline; there is no separate modelserver/xpu/ overlay specific to Flow Control because none is needed.

Prerequisites

- Chapter 0 (Common Installation) complete.
- No additional cluster prerequisites beyond Chapter 0.

Deployment Steps

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
export GUIDE_NAME="flow-control"
export NAMESPACE="llm-d-flow-control"
export MODEL_NAME="Qwen/Qwen3-32B" # swap for the CI-sized Qwen/Qwen3-0.6B
    ↪ on constrained Intel XPU test nodes

kubectl create namespace ${NAMESPACE} --dry-run=client -o yaml | kubectl apply
    ↪ -f -

# HF token secret (see Chapter 0.4)
kubectl create secret generic llm-d-hf-token \
  --from-literal="HF_TOKEN=${HF_TOKEN}" \
  --namespace "${NAMESPACE}" --dry-run=client -o yaml | kubectl apply -f -

# 1. Deploy the Router (Standalone Mode)
helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/${GUIDE_NAME}.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}

# 2. Deploy the Model Server – reuse the Optimized Baseline Intel XPU overlay,
#   relabeled to this guide's name via `sed` (this is the pattern the guide
    ↪ itself
#   prescribes for every non-default accelerator, not an Intel-XPU-specific
    ↪ workaround)
kubectl kustomize ${REPO_ROOT}/guides/optimized-baseline/modelserver/xpu/vllm/
    ↪ \
    | sed "s/optimized-baseline/${GUIDE_NAME}/g" \
    | kubectl apply -n ${NAMESPACE} -f -
```

For Gateway Mode (Kubernetes Gateway rather than standalone Envoy sidecar), see guides/flow-control/README.md “Gateway Mode” — the pattern is identical to the one used in Chapter 1.

Verification

```
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
    ↪ jsonpath='{.spec.clusterIP}')
kubectl logs deploy/${GUIDE_NAME}-epp -n ${NAMESPACE} | grep "Flow Control
    ↪ enabled"
```

Inference Test

Apply the three priority-tier Inference0bjectives shipped with the guide, then send a request tagged with a tenant/priority header:

```
kubectl apply -f ${REPO_ROOT}/guides/${GUIDE_NAME}/objectives.yaml -n
    ↪ ${NAMESPACE}
```

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
```

```

--namespace="${NAMESPACE}" \
--env="IP=$IP" --env="NAMESPACE=$NAMESPACE" --env="GUIDE_NAME=$GUIDE_NAME"
↪ \
-- /bin/bash

# from inside the debug pod:
curl -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-H 'x-llm-d-inference-fairness-id: tenant-a' \
-H 'x-llm-d-inference-objective: premium-traffic' \
-d "{\"model\": \"${MODEL_NAME}\", \"prompt\": \"Say hello\"}"

# confirm queuing metrics are exposed
curl http://${GUIDE_NAME}-epp:9090/metrics | grep
↪ llm_d_epp_flow_control_queue_size

```

To actually observe backpressure (not just admission), drive concurrent load with hey per the guide's "Use Case 2: Backpressure Management" section — a single request always dispatches immediately because the system is work-conserving.

Troubleshooting

- **Trust boundary:** never let end users self-assert `x-llm-d-inference-fairness-id` / `x-llm-d-inference-objective` directly — a production ingress Gateway must strip these headers from external traffic and re-inject them after authenticating the caller (see the EPP HTTP headers reference in docs/api-reference/epp-http-headers.md).
- If "Flow Control enabled" never appears in the EPP logs, confirm the `${GUIDE_NAME}.values.yaml` router overlay was actually applied (not just the base values file).
- `utilization-detector` (the default saturation detector) can lag under bursty Intel XPU telemetry collection intervals; switch to `concurrency-detector` per the Tuning Guide if you see saturation detected later than expected.

Cleanup

```

helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
# Must reuse the same kustomize+sed rename pipeline as the deploy step above –
↪ the resources were
# created under the ${GUIDE_NAME}-prefixed names, not the original
↪ optimized-baseline names, so a
# plain `kubectl delete -k .../optimized-baseline/modelserver/xpu/vllm/` would
↪ not match them.
kubectl kustomize ${REPO_ROOT}/guides/optimized-baseline/modelserver/xpu/vllm/
↪ \
| sed "s/optimized-baseline/${GUIDE_NAME}/g" \
| kubectl delete -n ${NAMESPACE} -f -
kubectl delete namespace ${NAMESPACE}

```

Config Reference

Field	Value	Notes
Default model (inherited)	Qwen/Qwen3-32B	8 replicas × TP2 = 16 GPUs in the reference config; scale down for Intel XPU test clusters
Fairness policy	global-strict-fairness-policy	ignores flow isolation, single global FCFS order
Ordering policy	fcfs-ordering-policy	first-come-first-served
Saturation detector	utilization-detector (default) / concurrency-detector (recommended for production)	
Priority tiers (example)	Premium (100), Standard (0), Best-Effort (-10)	defined in objectives.yaml
Client headers	x-llm-d-inference-fairness-id, x-llm-d-inference-objective	must be stripped/re-injected by ingress Gateway in production

2.2 Predicted Latency-Based Routing

Overview

Hardware-agnostic at the routing layer; the GPU/TPU-specific model-server paths shown upstream do not include an Intel XPU variant of their own — Intel XPU deployers reuse the Optimized Baseline overlay exactly as the guide’s “For other backends” note instructs.

Routes each inference request to the model server predicted to serve it fastest, using a live-trained XGBoost latency-prediction model instead of heuristic queue-depth/KV-utilization scoring — and, optionally, only to a server predicted to meet a per-request TTFT/TPOT SLO. Source: guides/predicted-latency-routing/README.md.

When to pick this path: workloads with high variance in prompt/completion length where queue depth alone is a poor load proxy, or where clients can express per-request latency SLOs you want the gateway to enforce. **Skip it** for heterogeneous pools (mixed GPU types/model variants) — the predictor assumes a single pod shape and will produce inaccurate predictions otherwise.

[!NOTE] Upstream flags OpenShift support for this guide as “not reliable as-is” — the latency-predictor sidecars may need additional OpenShift-specific runtime adjustments. Prefer a vanilla Kubernetes distribution for your first Intel XPU validation pass.

Prerequisites

- Chapter 0 (Common Installation) complete.
- No additional cluster prerequisites beyond Chapter 0.

Deployment Steps

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
```

```

export GUIDE_NAME="predicted-latency-routing"
export NAMESPACE=llm-d-predicted-latency
export MODEL_NAME="Qwen/Qwen3-32B"

kubectl create namespace ${NAMESPACE}
kubectl create secret generic llm-d-hf-token \
  --from-literal="HF_TOKEN=${HF_TOKEN}" \
  --namespace "${NAMESPACE}" --dry-run=client -o yaml | kubectl apply -f -

# 1. Deploy the Router – default values file trains the predictor on
  ↳ end-to-end latency
#   (routing only, no SLO headers). Swap in
  ↳ router/predicted-latency-slo.values.yaml
#   for SLO-aware scheduling.
helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/${GUIDE_NAME}/router/predicted-latency.values.yaml
  ↳ \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}

# 2. Deploy the Model Server – Intel XPU is not one of this guide's own
  ↳ model-server
#   variants (only GPU/vLLM base+gke, and a TPU variant for the agentic case
  ↳ exist under
#   guides/predicted-latency-routing/modelserver/). Per the guide's own
  ↳ instructions for
#   "other backends", reuse the Optimized Baseline Intel XPU overlay directly
  ↳ – both
#   values files already select pods labeled
  ↳ `llm-d.ai/guide=optimized-baseline`, so no
#   relabeling `sed` step is required here (unlike Flow Control in Chapter
  ↳ 2.1).
kubectl apply -n ${NAMESPACE} -k
  ↳ ${REPO_ROOT}/guides/optimized-baseline/modelserver/xpu/vllm/

```

Verification

Confirm predictions are actually being produced in Prometheus (see docs/architecture/advanced/latency-predictor.md#observability for the full metric reference):

`inference_objective_request_ttft_prediction_duration_seconds`

If this stays empty, the predictor sidecar isn't being called — tail the EPP logs for predicted-latency-producer errors.

Inference Test

```

export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o
  ↳ jsonpath='{.spec.clusterIP}')

kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace="${NAMESPACE}" \

```

```

--env="IP=$IP" --env="NAMESPACE=$NAMESPACE" --env="MODEL_NAME=$MODEL_NAME"
↪ \
-- /bin/bash

# from inside the debug pod – opt this request into SLO-aware routing:
curl -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-H 'x-llm-d-slo-ttft-ms: 200' \
-H 'x-llm-d-slo-tpot-ms: 50' \
-d '{
  "model": "'${MODEL_NAME}'",
  "prompt": "Explain the difference between prefill and decode.",
  "max_tokens": 200,
  "temperature": 0,
  "stream": true,
  "stream_options": {"include_usage": true}
}'

```

No header changes are needed for plain latency-based routing — it applies to every request. SLO headers (`x-llm-d-slo-ttft-ms`, `x-llm-d-slo-tpot-ms`) opt a request into SLO enforcement; sheddable (negative-priority) requests are rejected at admission if no endpoint can meet the SLO, rather than routed to a guaranteed miss.

Troubleshooting

- **Empty prediction metrics:** predictor sidecar not being invoked — check EPP logs for predicted-latency-producer errors, and confirm the router values file actually enabled the predictor plugin (not just the base values file).
- **Predictions drifting from reality:** compare `inference_objective_request_predicted_ttft_seconds` against `inference_objective_request_ttft_seconds` over a rolling window — a healthy deployment converges within a few percent after warmup; if it doesn't, the pool is likely heterogeneous (mixed hardware/model shapes), which this guide explicitly does not support.
- **OpenShift:** known-unreliable per upstream notes — try a vanilla Kubernetes distro first if you hit sidecar runtime issues.

Cleanup

```

helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}
kubectl delete -n ${NAMESPACE} -k
↪ ${REPO_ROOT}/guides/optimized-baseline/modelserver/xpu/vllm/
kubectl delete namespace ${NAMESPACE}

```

Config Reference

Field	Value	Notes
Values file (default)	<code>router/predicted-latency.values.yaml</code>	routing-only, predictor trained on E2E latency
Values file (SLO-aware)	<code>router/predicted-latency-slo.values.yaml</code>	requires <code>"stream": true</code> on every request
SLO headers	<code>x-llm-d-slo-ttft-ms</code> , <code>x-llm-d-slo-tpot-ms</code>	opt-in per request

Field	Value	Notes
Model server (Intel XPU)	reuses guides/optimized-baseline/modelserver/xpu/v1flow directly	no relabeling needed, unlike V1flow Control
Key metrics	inference_objective_requests_per_minute, ..._predicted_ttft_seconds, ..._ttft_slo_violation_total	see architecture doc for full list

2.3 Workload Autoscaling

Overview

Hardware-agnostic — scales whatever model-server Deployment you already run (including the Intel XPU Optimized Baseline overlay); no accelerator-specific autoscaling logic exists.

CPU/GPU utilization metrics are poor autoscaling signals for LLM inference — accelerators often sit near 100% “utilized” during active batching regardless of actual load. This guide covers two proactive, SLO-aware autoscaling paths built on real inference-demand signals (queue depth, in-flight requests, KV-cache pressure). Source: guides/workload-autoscaling/README.md.

Path	Signal source	Best for
KEDA + EPP Metrics (README.hpa-epp.md)	EPP-emitted queue depth / running-request-count metrics	Homogeneous hardware, each model-server pool scaled independently
HPA + WVA Metrics (README.wva.md)	Workload Variant Autoscaler’s wva_desired_replicas (KV utilization, queue depth, perf budgets)	Multi-variant deployments across heterogeneous hardware, cost-aware scaling

An experimental **Replica Rebalancing** feature (README.replica-rebalancing.md) adjusts max replica counts across annotated HPAs sharing a GPU budget — explicitly flagged upstream as a proof-of-concept, not production-ready.

KEDA (or the OpenShift Custom Metrics Autoscaler Operator) is the recommended metrics adapter for both paths — Prometheus Adapter is deprecated. The **KEDA + EPP Metrics** path documents KEDA as the primary supported route. The **HPA + WVA Metrics** path can use KEDA-managed HPA (recommended) or a plain HPA reading the wva_desired_replicas external metric directly, without KEDA.

Prerequisites

- Chapter 0 (Common Installation) complete, plus a running **Prometheus** instance (see docs/operations/observability/setup.md) — WVA requires TLS enabled on Prometheus.
- **KEDA** installed (recommended path) — see the KEDA install guide. On OpenShift, use the Custom Metrics Autoscaler Operator instead.
- Complete Optimized Baseline on Intel XPU first, **including its optional monitoring step** — this guide layers on top of it and needs the EPP metrics endpoint already being scraped.

Deployment Steps (KEDA + EPP Metrics path)

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
export NAMESPACE=llm-d-optimized-baseline
export MONITORING_NAMESPACE=llm-d-monitoring
export KEDA_NAMESPACE=keda

# Upgrade the Optimized Baseline router with the KEDA+EPP overlay (enables EPP
  ↳ Flow Control)
# – reapply the monitoring feature values so the EPP metrics
  ↳ port/ServiceMonitor stay enabled
helm upgrade optimized-baseline \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/optimized-baseline/router/optimized-
    ↳ baseline.values.yaml \
  -f ${REPO_ROOT}/guides/recipes/router/features/monitoring.values.yaml \
  -f ${REPO_ROOT}/guides/workload-autoscaling/keda-epp/router.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

The rest of the KEDA ScaledObject setup (targeting `llm_d_epp_flow_control_queue_size` and `llm_d_epp_request_running` via PromQL) is identical regardless of the accelerator backing the model-server Deployment — see `guides/workload-autoscaling/README.hpa-epp.md` for the full ScaledObject manifest and PromQL queries.

For the WVA path (multi-variant, cost-aware scaling across heterogeneous hardware), see `guides/workload-autoscaling/README.wva.md` — most relevant if you run Intel XPU alongside other accelerator types and want WVA to prefer the cheaper variant automatically.

Verification

```
kubectl logs deployment/optimized-baseline-epp -n ${NAMESPACE} | grep "Flow
  ↳ Control enabled"
kubectl get servicemonitor -n ${NAMESPACE}

# confirm EPP exposes the metrics directly
kubectl port-forward -n ${NAMESPACE} service/optimized-baseline-epp 9091:9090
  ↳ &
curl -s http://localhost:9091/metrics | grep -E
  ↳ 'llm_d_epp_flow_control_queue_size|llm_d_epp_request_running'
```

Then, in your Prometheus query UI, confirm the metric is actually being scraped:

```
sum(llm_d_epp_flow_control_queue_size{namespace="llm-d-optimized-
  ↳ baseline",service="optimized-baseline-epp",model_name="Qwen/Qwen3-32B"})
```

Inference Test

Drive sustained concurrent load against the Optimized Baseline endpoint (see Optimized Baseline's Inference Test) and watch the ScaledObject's managed HPA add replicas as queue depth/running-request-count rise, then scale back down as load subsides.

Troubleshooting

- **Do not create a separate HPA** for a Deployment already managed by a KEDA ScaledObject — two HPAs targeting the same Deployment produce conflicting scaling decisions. KEDA's generated HPA remains visible for inspection only.
- If metrics never populate in Prometheus, confirm the ServiceMonitor survived the router helm upgrade in the deployment step — reapplying the KEDA+EPP overlay without also re-including the monitoring values file will silently disable metric scraping.
- WVA requires TLS on the Prometheus endpoint; if the WVA controller can't reach Prometheus, check certificate/auth configuration first.

Cleanup

```
# Remove the KEDA ScaledObject (see README.hpa-epp.md for its exact name)
kubectl delete scaledobject <scaledobject-name> -n ${NAMESPACE}

# Revert the router to the plain Optimized Baseline values (drop the keda-epp
↪ overlay)
helm upgrade optimized-baseline \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/optimized-baseline/router/optimized-
↪ baseline.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

Config Reference

Field	Value	Notes
Scaling signal (KEDA+EPP)	llm_d_epp_flow_control_queue_size	EPP-native metrics
Scaling signal (WVA)	llm_d_epp_request_running wva_desired_replicas	aggregates KV utilization + queue depth + perf budget
Scale-to-zero	Supported on both paths	
Additional components (KEDA path)	KEDA, Prometheus	no Prometheus Adapter needed
Additional components (WVA path)	WVA controller, Prometheus (TLS required)	
Replica Rebalancing	experimental, proof-of-concept only	not production-ready per upstream

2.4 Rollouts

Repo location note: the guides-index (guides/README.md) links to ./rollouts/README.md, but that path has moved to docs/operations/rollouts/ as part of a broader documentation reorganization (“Reorganize part 2: create two new pillars, operations and infrastructure”). The commands below reference the current docs/operations/rollouts/ path.

Overview

Hardware-agnostic — directly useful for migrating an existing NVIDIA deployment to Intel XPU (or vice versa) with minimal disruption, see “Node/Accelerator Update Roll Out” below.

Rollout guides cover three incremental deployment strategies for updating inference infrastructure with minimal service disruption. Source: docs/operations/rollouts/README.md.

Strategy	Mechanism	Deployment mode	Best for
Rolling Update	Standard Kubernetes Deployment rolling update (e.g. 25% at a time)	Standalone or Gateway	General, non-critical updates; conserves compute
Blue-Green Update	Second complete InferencePool + HTTPRoute traffic-weight splitting	Gateway only	Critical production rollouts, fast rollback, canary by header
LoRA Adapter Rollout	InferenceModelRewrite mapping model names to adapter versions	Standalone or Gateway	Updating LoRA adapters without touching base model/infra

Why this matters for Intel XPU specifically: Blue-Green Update’s documented use case #1 is literally “**Node(compute, accelerator) update roll out**” — safely migrating inference workloads to new node hardware or accelerator configurations without interrupting service. This is the recommended mechanism for migrating an existing NVIDIA InferencePool to Intel XPU nodes incrementally (e.g. 1% → 10% → 50% → 100% traffic shift) rather than a hard cutover.

Prerequisites

- Chapter 0 (Common Installation) complete, with a working deployment already running (e.g. Optimized Baseline on Intel XPU) that you intend to update/migrate.
- For Blue-Green Update: llm-d Router in **Gateway mode** (a Kubernetes Gateway + HTTPRoute) — see docs/infrastructure/gateway/; Blue-Green does not work in standalone mode.

Deployment Steps

Rolling Update

Standard Kubernetes mechanics — update the vLLM image/tag or config in your existing Helm values/Kustomize overlay and re-apply; Kubernetes replaces pods incrementally while old pods keep serving traffic. No llm-d-specific steps beyond your normal `helm upgrade / kubectl apply -k`.

Blue-Green Update — example: Intel XPU node/accelerator migration

Starting from an existing InferencePool (e.g. vllm-qwen3-32b on NVIDIA GPU nodes):

1. Deploy a second, complete InferencePool (the "green" pool) on Intel XPU nodes,

```

#   with a different Helm release name / selector – e.g. by applying the
↪   Optimized
#   Baseline Intel XPU overlay under a new name:
kubectl kustomize ${REPO_ROOT}/guides/optimized-baseline/modelserver/xpu/vllm/
↪ \
| sed "s/optimized-baseline/vllm-qwen3-32b-new/g" \
| kubectl apply -n ${NAMESPACE} -f -

# 2. Edit the HTTPRoute to split traffic between old (blue, NVIDIA) and new
↪ (green, Intel XPU)
kubectl edit httproute llm-route

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: llm-route
spec:
  parentRefs:
    - group: gateway.networking.k8s.io
      kind: Gateway
      name: inference-gateway
  rules:
    - backendRefs:
        - group: inference.networking.k8s.io
          kind: InferencePool
          name: vllm-qwen3-32b           # old (blue)
          weight: 90
        - group: inference.networking.k8s.io
          kind: InferencePool
          name: vllm-qwen3-32b-new      # new (green, Intel XPU)
          weight: 10
      matches:
        - path:
            type: PathPrefix
            value: /

```

Gradually increase the green weight (10 → 50 → 100) as you validate correctness/performance on Intel XPU, then finish the rollout by setting the green pool's weight to 100 and removing the backendRefs entry for the old pool entirely. Roll back instantly at any point by flipping the weights back.

LoRA Adapter Rollout

See docs/operations/rollouts/adapter-rollout.md — uses InferenceModelRewrite to gradually shift traffic between adapter versions without any infrastructure change. Fully accelerator agnostic; identical on Intel XPU.

Verification

```

IP=$(kubectl get gateway/inference-gateway -o
↪ jsonpath='{.status.addresses[0].value}'); PORT=80

# send a batch of requests and confirm the observed split roughly matches the
↪ configured weights
for i in $(seq 1 20); do

```

```
curl -s -o /dev/null -w "%{http_code}\n" ${IP}:${PORT}/v1/completions \
-H 'Content-Type: application/json' \
-d '{"model": "vllm-qwen3-32b", "prompt": "ping", "max_tokens": 1}'
```

done

Cross-check which pool actually served each request via response headers or per-pool EPP logs.

Inference Test

```
curl -i ${IP}:${PORT}/v1/completions -H 'Content-Type: application/json' -d '{
  "model": "vllm-qwen3-32b",
  "prompt": "Write as if you were a critic: San Francisco",
  "max_tokens": 100,
  "temperature": 0
}'
```

Troubleshooting

- Blue-Green **requires Gateway mode** — attempting this in standalone mode has no HTTPRoute to edit; use Rolling Update or LoRA Adapter Rollout instead if you're on standalone.
- Keep the old (blue) InferencePool and its nodes running until the new (green) pool is fully validated — this is your rollback path; deleting it early forfeits instant rollback.
- If traffic doesn't appear to split according to configured weights, confirm both InferencePools are correctly referenced in backendRefs and that neither pool's own Helm chart was installed with httpRoute.create=true (a pool-level catch-all route will conflict with this guide's manually-edited HTTPRoute, the same conflict called out in Multi-Model Routing).

Cleanup

Once fully migrated to the new (green) pool, remove the old one:
 helm uninstall vllm-qwen3-32b -n \${NAMESPACE}

Config Reference

Field	Value	Notes
Strategy	Blue-Green (HTTPRoute weight-based traffic split)	Gateway mode only
backendRefs[].weight	integer, relative	e.g. 90/10 → 50/50 → 0/100
Rollback	flip weights back to 100/0	instant, no pod recreation needed
Alternative strategy	Rolling Update	standalone + gateway, in-place pod replacement
Alternative strategy	LoRA Adapter Rollout	InferenceModelRewrite, no infra change

3.1 Agentic Serving

Overview

This is a **workload composition guide**, not a standalone deployable overlay — its two ready-made hardware-specific deployments (NVIDIA H200, Google TPU v7) do not include an Intel XPU variant; Intel XPU users compose the same building blocks manually as described below.

Agentic Serving is a horizontal, workload-centric umbrella guide for serving agentic *programs* (e.g. coding agents) on llm-d — it composes several capability guides into one recommended, cohesive deployment rather than being a single new feature. Source: [guides/agentic-serving/README.md](#).

The reference workload is **long-horizon loops** (agentic code generation): deep multi-turn sessions over large, repository-scale contexts with tool-call pauses between turns. Three behaviors drive the design: prefill-heavy/decode-light (large context dominates TTFT), high reusable locality (cache hit rate — not FLOPs — sets throughput), and bursty/stateful arrivals (tool pauses leave sessions idle, then resume in bursts).

The optimization stack (each layer already covered elsewhere in this whitepaper):

Layer	Chapter	What it does for the workload
Optimized Baseline	1.1	Prefix-cache scorer routes a turn to the replica already holding its prefix
Tiered Prefix Cache	1.4	Offloads KV cache beyond accelerator memory so idle sessions restore on resume instead of recomputing prefill
Precise Prefix Cache Routing	1.3	Exact, global cache-state view enabling session-centric orchestration
P/D Disaggregation	1.2	Separates prefill/decode pools so heavy prefill never stalls token generation

Upstream ships two **ready-made** deployments composing this stack, neither of which targets Intel XPU:

- `nemotron-3-ultra-550b-h200.md` — P/D-disaggregated serving on 8× NVIDIA H200 with CPU KV-offloading and ready-to-use coding-agent client configs.
- `qwen3-coder-480b-tpu.md` — routing + CPU KV-offloading on 8× Google TPU v7x (2x2x1).

Prerequisites

- Chapter 0 (Common Installation) complete.
- Each of the four composed guides' own prerequisites (Chapters 1.1–1.4 of this whitepaper), since Agentic Serving is additive on top of them rather than a replacement.

Deployment Steps (Intel XPU — compose the building blocks manually)

Since no ready-made Intel XPU deployment ships upstream, assemble the stack yourself from the already-validated Chapter 1 building blocks, following the same pattern as the two upstream examples (routing + KV-offloading, optionally + P/D disaggregation for larger models):

```

# 1. Deploy P/D Disaggregation on Intel XPU as the serving foundation for a
  ↪ large model
#   (see Chapter 1.2 for full steps) – this gives you separate prefill/decode
  ↪ pools.

# 2. Layer Tiered Prefix Cache's KV-offloading connector on top of the same
  ↪ model server
#   (see Chapter 1.4) so idle agentic sessions can resume from offloaded KV
  ↪ state instead of
#   recomputing the full prompt prefix.

# 3. Deploy the router with BOTH the P/D and tiered-prefix-cache values files
  ↪ layered together
#   (adjust chart/values paths to match your actual guide combination):
helm install agentic-xpu \
  ${ROUTER_STANDALONE_CHART} \
  -f ${REPO_ROOT}/guides/recipes/router/base.values.yaml \
  -f ${REPO_ROOT}/guides/pd-disaggregation/router/pd-
  ↪ disaggregation.values.yaml \
  -f ${REPO_ROOT}/guides/tiered-prefix-cache/router/tiered-prefix-
  ↪ cache.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}

Caveat: this composition has not been validated by upstream CI for Intel XPU (unlike
the individual Chapter 1 guides, which do have Intel XPU CI or documented validated
configs). Treat this as a starting point requiring your own end-to-end validation, not
a tested recipe.

```

Verification

Reuse each composed guide's own verification steps (P/D disaggregation pod health, tiered prefix-cache offload connector health) — see Chapters 1.2 and 1.4.

Inference Test

Use a realistic agentic-style prompt sequence (large repository-scale context, multiple turns reusing the same prefix) rather than a single short completion, since the whole point of this composition is prefix-cache reuse across turns:

```

curl -X POST http://${IP}/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "${MODEL_NAME}", "messages": [
    {"role": "system", "content": "<large repository context>"},
    {"role": "user", "content": "Refactor this function to be async"}
  ], "max_tokens": 500}'

```

Troubleshooting

- If cache-reuse benefits don't materialize, confirm Precise Prefix Cache Routing's `--block-size` matches the router's `tokenProcessorConfig.blockSize` exactly (both default 64 — see Chapter 1.3) — a mismatch silently degrades cache-aware scoring rather than erroring out.
- If offloaded sessions don't resume correctly, check that Tiered Prefix Cache's `securityContext.privileged: true` requirement (Chapter 1.4) is actually satisfied under your cluster's Pod Security Admission policy.

Cleanup

```
helm uninstall agentic-xpu -n ${NAMESPACE}
# then clean up each composed guide's model-server overlay per its own Chapter
  ↳ 1 cleanup steps
```

Config Reference

Field	Value	Notes
Composed layers	Optimized Baseline + Tiered Prefix Cache + Precise Prefix Cache Routing + P/D Disaggregation	
Ready-made deployments (upstream)	NVIDIA H200 (Nemotron-3-Ultra-550B), Google TPU v7 (Qwen3-Coder-480B)	neither targets Intel XPU
Intel XPU status	Manual composition of Chapter 1 building blocks	not upstream-CI-validated as a combined stack
Benchmark harness	inference-perf via llm-d-benchmark, replaying agentic-style traces	see guide for exact preset

3.2 Multi-Model Routing

Overview

Hardware-agnostic — the Inference Payload Processor (IPP) and HTTPRoute-based routing operate purely at the Gateway layer; each backing InferencePool can independently be an Intel XPU deployment.

Deploys the **Inference Payload Processor (IPP)** to serve multiple LLMs behind a single Gateway endpoint. IPP extracts the model name from the request body and sets routing headers; HTTPRoutes then match those headers to direct traffic to the correct InferencePool. Source: guides/multi-model-routing/README.md.

Use this guide when you need to serve multiple base models (e.g. Qwen for chat, DeepSeek for reasoning) behind one OpenAI-compatible endpoint. For a single model, use Optimized Baseline instead.

Prerequisites

- Chapter 0 (Common Installation) complete.
- **Multiple InferencePools already deployed**, each serving a different base model — follow Optimized Baseline independently for each pool you want to add (e.g. once on Intel XPU for Qwen, once for a second model). When deploying each pool for this guide, do **not** pass `--set httpRoute.create=true` — this guide's own HTTPRoutes (deployment step 3) handle model-name-header-based routing, and a pool-level catch-all route would conflict.
- A Kubernetes Gateway (Istio, GKE, AgentGateway, etc.) — see docs/infrastructure/gateway/README.

Deployment Steps

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
export GUIDE_NAME="multi-model-routing"
export NAMESPACE="llm-d-multi-model"

kubectl create namespace ${NAMESPACE} --dry-run=client -o yaml | kubectl apply
↪ -f -

# 1. Deploy IPP (clone its own repo; it is not part of the llm-d repo)
git clone https://github.com/llm-d/llm-d-inference-payload-processor.git
↪ /tmp/ipp
helm install ipp /tmp/ipp/config/charts/payload-processor \
  --set provider.name=istio \
  --set inferenceGateway.name=llm-d-inference-gateway \
  --set payloadProcessor.image.tag=v0.1.0-rc.4 \
  -n ${NAMESPACE}
# --set provider.name=gke on GKE. --set provider.name=none skips the
↪ auto-generated
# Istio EnvoyFilter/GKE GCPRoutingExtension – a Gateway is still required
↪ either way, you'd just
# wire IPP's ext-proc integration into it yourself instead of letting the
↪ chart do it.

kubectl get pods -n ${NAMESPACE} -l app=payload-processor

# 2. Register each base model with IPP via ConfigMap (edit
↪ manifests/configmaps.yaml to match
# your actual Intel XPU model names/pools first)
kubectl apply -n ${NAMESPACE} -f
↪ ${REPO_ROOT}/guides/multi-model-routing/manifests/configmaps.yaml
# each ConfigMap needs label inference.llm-d.ai/ipp-managed: "true" and a
↪ baseModel matching
# your Intel XPU pool's actual served model name

# 3. Configure HTTPRoutes matching IPP's injected X-Gateway-Base-Model-Name
↪ header
# (edit manifests/httproutes.yaml: parentRefs -> your Gateway, backendRefs
↪ -> your pool names)
kubectl apply -n ${NAMESPACE} -f
↪ ${REPO_ROOT}/guides/multi-model-routing/manifests/httproutes.yaml
```

Verification

```
kubectl logs -n ${NAMESPACE} -l app=payload-processor --tail=100
kubectl get configmap -l inference.llm-d.ai/ipp-managed=true -n ${NAMESPACE}
# confirm requests reach the correct pool (replace <pool-name> with your
↪ InferencePool name)
kubectl logs -n ${NAMESPACE} -l llm-d-router-gateway=<pool-name>-epp --tail=20
```

Inference Test

```
export GATEWAY_IP=$(kubectl get gateway llm-d-inference-gateway -n
↪ ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}')
```

```
# Request routed to your first (e.g. Intel XPU Qwen) pool
curl -X POST "http://${GATEWAY_IP}/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -d '{"model": "Qwen/Qwen3-32B", "messages": [{"role": "user", "content":
    ↪ "Hello"}]}'

# Request routed to a second pool
curl -X POST "http://${GATEWAY_IP}/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -d '{"model": "deepseek/DeepSeek-r1", "messages": [{"role": "user",
    ↪ "content": "Solve this problem"}]}'
```

Advanced: LoRA Adapter Routing

Once base model routing works, extend a ConfigMap with an adapters field to route LoRA adapter names to their base model's pool:

```
kubectl apply -n ${NAMESPACE} -f - <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  name: qwen-model-mapping
  labels:
    inference.llm-d.ai/ipp-managed: "true"
data:
  baseModel: "Qwen/Qwen3-32B"
  adapters: |
    - food-review-1
    - travel-assistant
EOF
```

```
curl -X POST "http://${GATEWAY_IP}/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -d '{"model": "food-review-1", "messages": [{"role": "user", "content":
    ↪ "Review this restaurant"}]}'
```

All model and adapter names must be globally unique across all InferencePools/ConfigMaps.

Troubleshooting

- Requests routed to the wrong pool (or 404): confirm the ConfigMap's baseModel value matches the model name clients send exactly, and that the inference.llm-d.ai/ipp-managed: "true" label is present — IPP logs discovered model mappings at startup, check them first.
- If a pool's traffic bypasses IPP header-based routing entirely, verify that pool's Helm install did **not** include --set httpRoute.create=true (see Prerequisites) — its own catch-all route would out-compete this guide's header-matched HTTPRoutes.

Cleanup

```
kubectl delete -n ${NAMESPACE} -f
↪ ${REPO_ROOT}/guides/multi-model-routing/manifests/httproutes.yaml
kubectl delete -n ${NAMESPACE} -f
↪ ${REPO_ROOT}/guides/multi-model-routing/manifests/configmaps.yaml
```

```
helm uninstall ipp -n ${NAMESPACE}
rm -rf /tmp/ipp
kubectl delete namespace ${NAMESPACE}
```

Config Reference

Field	Value	Notes
Routing header	X-Gateway-Base-Model-Name	injected by IPP, matched by HTTPRoute
ConfigMap label	inference.llm-d.ai/ipp-managed: "true"	required for IPP discovery
Per-pool Helm flag	must NOT set	avoids catch-all route conflicts
LoRA routing	httpRoute.create=true adapters field in ConfigMap	maps adapter name -> base model's pool
IPP repo	github.com/llm-d/llm-d-inference-payload-processor	separate repo, not part of llmd

4.1 Asynchronous Processing

Overview

Hardware-agnostic — the Async Processor dispatches to whatever llm-d Router endpoint you give it; the backing model server can be any Optimized Baseline overlay, including Intel XPU.

The Async Processor processes inference requests asynchronously via a queue-based architecture — ideal for latency-insensitive workloads or filling idle accelerator capacity. It decouples submission from execution (clients submit to a queue, retrieve results later), optimizes resource utilization, and provides automatic retries without impacting real-time traffic. Source: [guides/asynchronous-processing/README.md](#).

Two supported queue implementations:

- **GCP Pub/Sub** ([gcp-pubsub/README.md](#)) — cloud-native, scalable messaging.
- **Redis Sorted Set** ([redis/README.md](#)) — high-performance, persisted, prioritized queue.

Prerequisites

- Kubernetes v1.31+ (Kind/Minikube for local dev; GKE/AKS/OpenShift for production).
- **Gateway Mode only:** a Gateway control plane deployed ([docs/infrastructure/gateway/README.md](#)). Not needed if you use Standalone Mode (step 1 below), which talks to the Router's Service clusterIP directly instead of a Gateway.
- An existing Optimized Baseline stack on Intel XPU to dispatch requests to — Async Processor is additive, not a replacement for the model-serving stack.

Deployment Steps

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
```

```
# 1. Get the IP of the llm-d Router deployed for Optimized Baseline (Chapter 1.1)
```

```

# Standalone Mode:
export IP=$(kubectl get service optimized-baseline-epp -n
↪ llm-d-optimized-baseline -o jsonpath='{.spec.clusterIP}')
# Gateway Mode:
export IP=$(kubectl get gateway llm-d-inference-gateway -n
↪ llm-d-optimized-baseline -o jsonpath='{.status.addresses[0].value}')

# 2. Choose a queue implementation and review/edit its values.yaml:
#   guides/asynchronous-processing/gcp-pubsub/values.yaml
#   guides/asynchronous-processing/redis/values.yaml

# 3. Deploy the Async Processor
export NAMESPACE=llm-d-async
export MQ_PROVIDER=redis # or gcp-pubsub
export ASYNC_VERSION=0.6.1

helm install async-processor \
  oci://ghcr.io/llm-d-incubation/charts/async-processor \
  -f ${REPO_ROOT}/guides/asynchronous-processing/${MQ_PROVIDER}/values.yaml
↪ \
  --set ap.igwBaseUrl=http://${IP}:80 \
  -n ${NAMESPACE} --create-namespace --version ${ASYNC_VERSION}

```

Verification

```

kubectl get pods -n ${NAMESPACE}
kubectl logs -n ${NAMESPACE} -l app.kubernetes.io/name=async-processor
↪ --tail=50

```

Inference Test

Testing steps are queue-implementation specific — follow the “Testing” section of whichever sub-guide you chose:

- Redis: guides/asynchronous-processing/redis/README.md#testing
- GCP Pub/Sub: guides/asynchronous-processing/gcp-pubsub/README.md#testing

Both follow the same submit-then-poll pattern: publish a request payload to the configured queue, then poll for the corresponding result once the Async Processor has dispatched it to the Intel XPU-backed Optimized Baseline endpoint and received a response.

Troubleshooting

- If the Async Processor never dispatches requests, confirm ap.igwBaseUrl actually resolves to a live llm-d Router endpoint (curl it directly first) — a common mistake is pointing at the wrong namespace’s Router IP.
- Redis is documented as sufficient for both development and production priority-queue needs; GCP Pub/Sub requires GCP-side IAM/topic setup not covered by the llm-d chart itself.

Cleanup

```

helm uninstall async-processor -n ${NAMESPACE}

```

Config Reference

Field	Value	Notes
Queue implementations	GCP Pub/Sub, Redis Sorted Set	choose one via MQ_PROVIDER
Dispatch target	ap.igwBaseUrl	must point at a live llm-d Router (any accelerator, incl. Intel XPU)
Chart	oci://ghcr.io/llm-d-incubation/charts/async-processor	separate incubation repo

4.2 Batch Gateway

Overview

Hardware-agnostic — Batch Gateway dispatches to an existing llm-d Router endpoint; the backing model server can be any Optimized Baseline overlay, including Intel XPU.

Batch Gateway provides an OpenAI-compatible Batch API (/v1/batches, /v1/files) for submitting, tracking, and managing large-scale batch inference jobs (up to 50,000 requests per job, configurable), designed to coexist with interactive workloads on shared infrastructure. Source: guides/batch-gateway/README.md.

Three components: an **API Server** (REST endpoints), a **Batch Processor** (pulls jobs from a priority queue, builds per-model execution plans, dispatches to the llm-d Router, writes output files), and a **Garbage Collector** (cleans up expired jobs/files).

Prerequisites

- Kubernetes v1.25+, Helm 3.0+.
- An existing Optimized Baseline stack on Intel XPU to dispatch requests to.
- **PostgreSQL 12+** for jobs/files metadata (Redis/Valkey dev-only alternative).
- **Redis 6+ / Valkey 8+** for the priority queue, events, and status updates.
- **S3 or a Filesystem PVC** for batch input/output file storage.

Deployment Steps

```
export NAMESPACE=batch-gateway
kubectl create namespace ${NAMESPACE}
```

1. Storage/queue credentials secret

```
kubectl create secret generic batch-gateway-secrets -n ${NAMESPACE} \
  --from-literal=redis-url="redis://redis-master.redis.svc.cluster.local:6379/0" \
  --from-literal=postgresql-url="postgresql://user:pass@postgresql.postgresql.svc.cluster.local:5432/batchgateway" \
  --from-literal=s3-secret-access-key="<your-s3-secret-key>"
```

2. Point the Batch Processor at your Intel XPU-backed llm-d Router

```
export INFERENCE_GW_URL="http://infra-inference-scheduling-inference-gateway-
↳ istio.llm-d-inference-scheduler.svc.cluster.local:80"
```

3. Deploy

```
helm install batch-gateway oci://ghcr.io/llm-d-incubation/charts/batch-gateway
```

```
↳ \
  -n ${NAMESPACE} \
  --set processor.config.globalInferenceGateway.url="${INFERENCE_GW_URL}" \
  --set "apiserver.config.batchAPI.passThroughHeaders={Authorization}" \
  --set global.fileClient.fs.pvcName="batch-gateway-pvc"
```

For per-model gateways (different Intel XPU pools per model) instead of one global gateway URL, use `processor.config.modelGateways` — see the Helm chart README.

Verification

```
kubectl get pods -n ${NAMESPACE} # expect apiserver, processor, gc all
```

↳ *Running*

```
kubectl port-forward -n ${NAMESPACE} svc/batch-gateway-apiserver 8081:8081 &
curl http://localhost:8081/health
```

batch-gateway-apiserver exposes two separate ports on the same Service: 8081 for the health/readiness endpoint above, and 8000 for the actual REST API (confirmed by the in-cluster URL `http://batch-gateway-apiserver:8000` used in the project's own benchmark manifests). The Inference Test below needs its own port-forward to 8000.

Inference Test

```
kubectl port-forward -n ${NAMESPACE} svc/batch-gateway-apiserver 8000:8000 &
```

1. Prepare a JSONL input file (OpenAI Batch API format)

```
cat > batch_input.jsonl <<'EOF'
{"custom_id": "req-001", "method": "POST", "url": "/v1/chat/completions",
↳ "body": {"model": "my-model", "messages": [{"role": "user", "content":
↳ "Hello"}], "max_tokens": 100}}
{"custom_id": "req-002", "method": "POST", "url": "/v1/chat/completions",
↳ "body": {"model": "my-model", "messages": [{"role": "user", "content":
↳ "What is llm-d?"}], "max_tokens": 200}}
EOF
```

2. Upload it

```
curl -X POST http://localhost:8000/v1/files -F "purpose=batch" -F
↳ "file=@batch_input.jsonl"
```

3. Create the batch job

```
curl -X POST http://localhost:8000/v1/batches \
  -H "Content-Type: application/json" \
  -d '{"input_file_id": "<file-id-from-upload>", "endpoint":
↳ "/v1/chat/completions", "completion_window": "24h"}'
```

4. Monitor

```
curl http://localhost:8000/v1/batches/<batch-id> | jq '{status,
↳ request_counts}'
```

5. Download results once status is "completed"

```

OUTPUT_FILE_ID=$(curl -s http://localhost:8000/v1/batches/<batch-id> | jq -r
  ↪ '.output_file_id')
curl http://localhost:8000/v1/files/${OUTPUT_FILE_ID}/content > results.jsonl

```

Troubleshooting

- `passThroughHeaders` must include any auth header (e.g. Authorization) your `llm-d` Router expects — the Batch Processor forwards these when dispatching individual requests; omitting a required header causes silent per-request auth failures inside batch jobs.
- For a production deployment with authentication, authorization, and TLS, see the Kubernetes deployment guide (Istio + Kuadrant + cert-manager) rather than the quick-start Helm install above.

Cleanup

```

helm uninstall batch-gateway -n ${NAMESPACE}
kubectl delete namespace ${NAMESPACE}

```

Config Reference

Field	Value	Notes
Max requests/job	50,000 (configurable)	
Metadata storage	PostgreSQL (prod) / Redis-Valkey (dev/test only)	
Queue/events storage	Redis/Valkey	
File storage	S3 or Filesystem PVC	
Dispatch target	<code>processor.config.globalInferenceGateway</code> or per-model <code>modelGateways</code>	any <code>BatchProcessor</code> , incl. Intel XPU
Related guide	Asynchronous Processing	per-request async queue; complementary to Batch Gateway's job-oriented API

5.1 No-Kubernetes Deployment

Overview

This guide targets NVIDIA GPUs + vLLM specifically — the model-server image, CLI flags, and `--gpu` container invocation shown upstream are NVIDIA-specific. It is **not** a maintained, first-class Intel XPU path today; Intel XPU support is a manual substitution (detailed below), not something upstream provides or tests.

Deploys the `llm-d` routing stack (EPP + Envoy + one or more vLLM workers) **without a Kubernetes cluster** — the EPP gets its endpoint inventory from a plain YAML file on disk via the file-discovery plugin instead of watching a Kubernetes `InferencePool`. Configs are plain YAML — no Helm chart, no Kustomize overlay; drop them on a host and run. Source: `guides/no-kubernetes-deployment/README.md`.

The EPP and Envoy configs are themselves **accelerator-agnostic** — only the model-server container invocation (image, `--gpu` flag) is NVIDIA-specific.

What Needs to Change for Intel XPU

Component	Upstream (NVIDIA)	Intel XPU substitution
Model server image	vllm/vllm-openai:v0.19.1	An Intel XPU-built vLLM image (VLLM_TARGET_DEVICE=xpu) — see the image-build notes referenced from Optimized Baseline / the equivalent Intel XPU vLLM container build process used for the Kustomize overlays in Chapter 1
GPU allocation flag	<code>docker run --gpus '"device=0,1"'</code>	Intel XPU has no direct Docker --gpus equivalent; instead expose the render/video device nodes: <code>docker run --device=/dev/dri:/dev/dri</code> and add the container user to the host's render/video groups (same GIDs used in the Kustomize overlays' commented-out securityContext.supplementalGroups, typically 44/991 — verify on your actual host with <code>getent group render video</code>)
Tensor-parallel comms backend	NCCL (--shm-size=20g for /dev/shm)	XCCL for Intel XPU multi-device collectives (see the Wide-EP guide for the XCCL note) — keep --shm-size sized similarly since vLLM's shared-memory IPC mechanism is backend-agnostic
EPP / Envoy config	unchanged	fully accelerator-agnostic, no changes needed

Prerequisites

- A host with Intel XPU device(s) exposed via /dev/dri, Docker (or Podman), and an Intel XPU vLLM image already built (see substitution table above).
- A HuggingFace token in HUGGING_FACE_HUB_TOKEN.
- Same environment setup as upstream:

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
export ENVOY_IMAGE=docker.io/envoyproxy/envoy:distroless-v1.33.2
export VLLM_IMAGE=<your-intel-xpu-vllm-image> # substitution – not the
↪ upstream default
```

```
export MODEL=Qwen/Qwen3-32B
```

Deployment Steps

1. Stage the configs

```
sudo mkdir -p /etc/epp /etc/envoy
sudo cp guides/no-kubernetes-deployment/router/epp/config.yaml
  ⇨ /etc/epp/config.yaml
sudo cp guides/no-kubernetes-deployment/router/epp/endpoints.yaml
  ⇨ /etc/epp/endpoints.yaml
sudo cp guides/no-kubernetes-deployment/router/envoy/envoy.yaml
  ⇨ /etc/envoy/envoy.yaml
```

2. Start the model server – Intel XPU substitution of the upstream `docker

```
  ⇨ run --gpus` example
docker run -d --name vllm-0 \
  --device=/dev/dri:/dev/dri \
  --group-add "$(getent group render | cut -d: -f3)" \
  --shm-size=20g \
  -p 8000:8000 \
  -e "HUGGING_FACE_HUB_TOKEN=${HUGGING_FACE_HUB_TOKEN}" \
  -e "VLLM_TARGET_DEVICE=xpu" \
  -v "${HOME}/.cache/huggingface:/root/.cache/huggingface" \
  --entrypoint vllm \
  "${VLLM_IMAGE}" \
  serve "${MODEL}" \
  --disable-access-log-for-endpoints=/health,/metrics,/v1/models \
  --tensor-parallel-size=2
```

```
until curl -sf http://127.0.0.1:8000/v1/models >/dev/null; do sleep 2; done
```

3. Edit /etc/epp/endpoints.yaml to list each worker (literal IPv4 addresses

```
  ⇨ required –
#   the file-discovery plugin does not resolve hostnames)
```

4. Start the EPP (accelerator-agnostic – unchanged from upstream)

```
docker run -d --name epp --network host \
  -v /etc/epp:/etc/epp:ro \
  "${ROUTER_EPP_IMAGE}:${ROUTER_EPP_VERSION}" \
  --config-file=/etc/epp/config.yaml \
  --pool-name=file-discovery --pool-namespace=default \
  --grpc-port=9002 --grpc-health-port=9003 --metrics-port=9090 \
  --secure-serving=false --v=2
```

5. Start Envoy (accelerator-agnostic – unchanged from upstream)

```
docker run -d --name envoy --network host \
  -v /etc/envoy/envoy.yaml:/etc/envoy/envoy.yaml:ro \
  "${ENVOY_IMAGE}" \
  --service-node envoy-proxy --log-level warn --concurrency 8 \
  --drain-strategy immediate --drain-time-s 60 \
  -c /etc/envoy/envoy.yaml
```

Verification

```
curl -s http://127.0.0.1:19000/ready
curl -s http://127.0.0.1:19000/clusters | grep -E
  ↳ '^(ext_proc|original_destination_cluster)'
curl -s http://127.0.0.1:9090/metrics | head
```

Inference Test

```
curl -s http://127.0.0.1:8081/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "Qwen/Qwen3-32B", "prompt": "How are you today?"}'
```

Troubleshooting

- **EPP not detecting endpoint changes:** confirm `watchFile: true` in `router/epp/config.yaml`; update `endpoints.yaml` via atomic rename (`mv endpoints.yaml.tmp endpoints.yaml`), and check docker logs `epp` for `endpoints` file changed, reloading.
- **Envoy returns 503:** verify EPP health (`curl http://127.0.0.1:9090/metrics`), confirm `endpoints.yaml` lists literal IPv4 addresses (not hostnames), and check the Envoy→EPP cluster (`curl http://127.0.0.1:19000/clusters | grep ext_proc`).
- **vLLM worker unreachable:** confirm the worker is up (`curl http://<worker-ip>:8000/v1/models`) and that the address in `endpoints.yaml` matches the worker's actual IP.
- **Intel XPU device not visible inside the container:** confirm `/dev/dri` is actually passed through (`--device=/dev/dri:/dev/dri`) and the container user is in the host's render group — a mismatched GID here is the most common first-run failure for this substitution.

Cleanup

```
docker rm -f envoy epp vllm-0
sudo rm -rf /etc/epp /etc/envoy
```

Config Reference

Field	Value	Notes
Default model (upstream) Discovery mechanism	Qwen/Qwen3-32B, TP=2 file-discovery plugin, <code>endpoints.yaml</code>	same as Optimized Baseline literal IPv4 addresses only, no DNS
Live reload	<code>watchFile: true</code>	atomic-rename updates to <code>endpoints.yaml</code> apply without EPP restart
P/D disaggregation outside Kubernetes	combine with P/D Disaggregation's EPP plugin config, set <code>llm-d.ai/role:</code> <code>prefill/decode</code> labels per endpoint	not Intel-XPU-specific, but untested in combination here
Hardware support status	NVIDIA + vLLM only, upstream-maintained	Intel XPU is a manual reader substitution (this chapter), not a maintained path

5.2 Encode Disaggregation (Multimodal)

Overview

`guides/multimodal-serving/e-disaggregation/` is an experimental guide deploying Qwen/Qwen3-VL-32B-Instruct with **encode disaggregation**: offloading the multimodal encoding stage (converting raw images/video/audio into embeddings) to dedicated Encode workers, consumed by prefill/decode workers alongside text tokens. Two topologies are supported: **E/PD** (Encode separated from combined Prefill+Decode) and **E/P/D** (full three-stage pipeline). Source: `guides/multimodal-serving/e-disaggregation/README.md`.

This is distinct from the **Multimodal Serving (Aggregated)** case already covered in Chapter 1.6, which *does* have a dedicated Intel XPU overlay. Encode Disaggregation is the more advanced, encode-stage-separated sibling of that guide.

Why this guide has no Intel XPU deployment steps

The guide’s own “Supported Hardware Backends” table lists exactly one backend:

Backend	Directory	Notes
NVIDIA GPU (vLLM)	<code>modelserver/gpu/vllm/</code>	vLLM with encode disaggregation

There is no `modelserver/xpu/` (or any other non-NVIDIA) entry. Unlike the router-layer/composite guides in Chapters 2-3 (which are hardware-agnostic and simply reuse whichever model-server overlay you deploy), Encode Disaggregation’s model-server manifests themselves are NVIDIA-specific, and there is no accelerator-agnostic substitution path documented upstream — unlike No-Kubernetes Deployment, where the substitution is straightforward.

If Intel XPU support is added upstream in the future, check `guides/multimodal-serving/e-disaggregation/` for a `modelserver/xpu/` overlay.

For a working, Intel-XPU-supported alternative today, see Multimodal Serving (Aggregated) — a simpler topology with a real `modelserver/xpu/vllm` overlay.

Appendix A. Environment Variable Reference

Variable	Defined in	Purpose
<code>REPO_ROOT</code>	<code>guides/env.sh</code>	Absolute path to the repo root
<code>GAIE_VERSION</code>	<code>guides/env.sh</code>	Gateway API Inference Extension CRD version
<code>ROUTER_CHART_VERSION</code>	<code>guides/env.sh</code>	llm-d Router chart version
<code>ROUTER_STANDALONE_CHART / ROUTER_GATEWAY_CHART</code>	<code>guides/env.sh</code>	Router chart OCI addresses
<code>NAMESPACE</code>	user-set	Target namespace
<code>HF_TOKEN</code>	user-set	HuggingFace access token
<code>GUIDE_NAME</code>	user-set, per case	Selects the values/overlay files for a given guide

Appendix B. Known Issues

- Routing proxy init container hang: `llm-d/llm-d#241` — workaround: `proxy.enabled: false`.
- vLLM usage-telemetry write failures under restricted SecurityContext (e.g. OpenShift): set `DO_NOT_TRACK=1`.